

Cadence — University Timetable App

Product & Technical Specification (v1.0)

Note on naming: "Cadence" is used as a placeholder project name throughout this document for readability. Replace with your chosen app name before publishing.

1. Document Overview

Purpose: This document defines the product scope, architecture, data model, feature set, UX flows, and platform integrations for Cadence — a native Android timetable app for tracking university class schedules.

Scope: Covers MVP (Phase 1) functionality in full detail, plus a specified set of expanded/modern features (Phase 2+) designed to be added without restructuring the core app.

Audience: Solo developer (you) or small team, acting as both product spec and engineering reference.

Guiding constraint: Every design and architecture decision below is made with one question in mind: *"Can this be extended later without a rewrite?"* Where a trade-off exists between "simplest to build now" and "easiest to extend later," this doc favors the latter, but flags where you can cut corners for a faster MVP.

2. Product Overview

2.1 Problem Statement

University students juggle recurring weekly class schedules that change every semester — different courses, teachers, rooms, and times. Generic calendar apps handle one-off events well but are clunky for "recurring weekly grid" schedules, don't cleanly separate semesters, and require an account/internet connection.

2.2 Target User

A university/college student who wants:

- A fast, glanceable view of "what do I have today / right now / next"
- An easy way to enter a semester's schedule once and forget about it
- Full offline reliability (no account, no network dependency)
- A clean, modern Android app that respects their device's look and feel (Material You)

2.3 Core Goals

1. Let the user build and view a weekly class timetable (day, time, course, teacher, room, notes).
2. Support easy semester-to-semester rollover (archive old, create new, optionally clone).
3. Work fully offline — no account, no server, no sync dependency required for core use.
4. Feel native to modern Android: Material You dynamic color, adaptive layouts, system-integrated features (widgets, notifications, shortcuts).
5. Be architected so new features (assignments, attendance, grades, exports, etc.) can be added as independent modules without touching the core scheduling engine.

2.4 Non-Goals (for MVP)

- No mandatory cloud account or login.
- No social/collaborative features (shared timetables between users).
- No school-specific integrations (LMS scraping, course catalog import) — may be considered far in the future as an optional import path, not a dependency.

2.5 Guiding Principles

- **Offline-first, always.** Every core feature must work with zero connectivity. Anything network-based (e.g., calendar export, optional cloud backup) is additive, never required.
 - **Local data ownership.** The user's schedule lives on their device by default; they can export/back it up manually.
 - **Progressive complexity.** The app should be dead simple for a student who just wants "day → class → room," while allowing power features (attendance, assignments, grades) to be opted into.
 - **Material You native.** Not just "supports dark mode" — full dynamic color, proper Material 3 components, and motion.
 - **Extensibility by structure, not by guesswork.** Achieved through clean architecture layers, a versioned data schema, and feature-boundary packages (detailed in Section 4).
-

3. Platform & Technical Requirements

Requirement	Value
Minimum SDK	API 31 (Android 12)
Target/Compile SDK	Latest stable at build time (API 35/36)
Language	Kotlin (100%)
UI Toolkit	Jetpack Compose
Design System	Material 3 (Material You)
Architecture	MVVM + Clean Architecture (UI / Domain / Data layers)
Local Database	Room (SQLite)
Async	Kotlin Coroutines + Flow
DI	Hilt
Navigation	Jetpack Navigation Compose
Preferences	Jetpack DataStore (Preferences)
Background Work	WorkManager
Widgets	Glance API (Compose-based widgets)
Min device form factors	Phone (primary), Tablet/Foldable (adaptive, Phase 2)

3.1 Key Library List

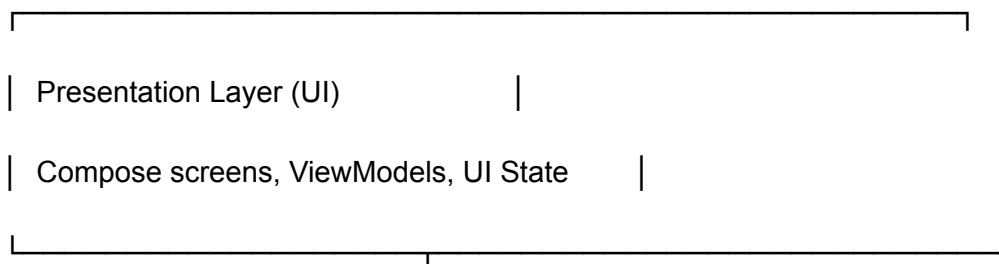
Purpose	Library
UI	<code>androidx.compose.material3</code> , <code>androidx.compose.ui</code>
Dynamic color	<code>androidx.compose.material3</code> (<code>dynamicLightColorScheme</code> / <code>dynamicDarkColorScheme</code>)

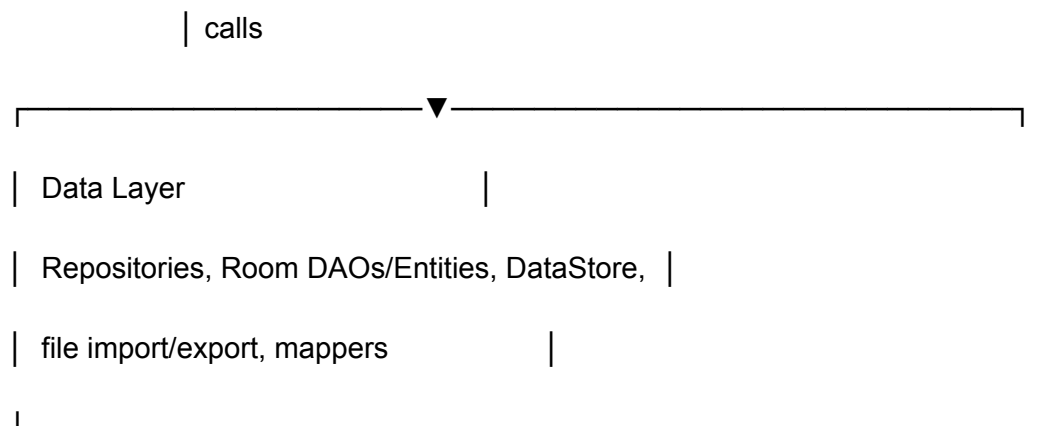
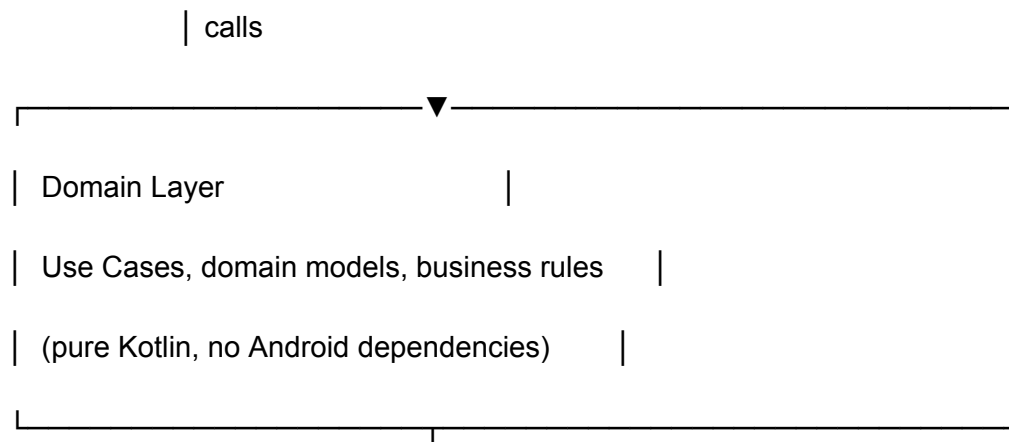
Purpose	Library
Navigation	<code>androidx.navigation:navigation-compose</code>
DI	<code>com.google.dagger:hilt-android</code>
Database	<code>androidx.room:room-runtime</code> , <code>room-ktx</code> , <code>room-compiler</code> (KSP)
Preferences	<code>androidx.datastore:datastore-preferences</code>
Background jobs	<code>androidx.work:work-runtime-ktx</code>
Widgets	<code>androidx.glance:glance-appwidget</code>
Date/time	<code>kotlinx-datetime</code> (preferred over <code>java.time</code> wrapper boilerplate, though <code>java.time</code> via desugaring is a valid alternative)
Image/icon loading (if needed)	<code>io.coil-kt:coil-compose</code>
Testing	JUnit5, Turbine (Flow testing), Compose UI Test, Robolectric (optional)

4. Architecture

4.1 Layered Structure

Cadence uses a **three-layer Clean Architecture**, even as a single Gradle module initially. This keeps the boundaries clean now so it's trivial to split into real Gradle modules later (`:core`, `:feature-timetable`, `:feature-assignments`, etc.) if the app grows.

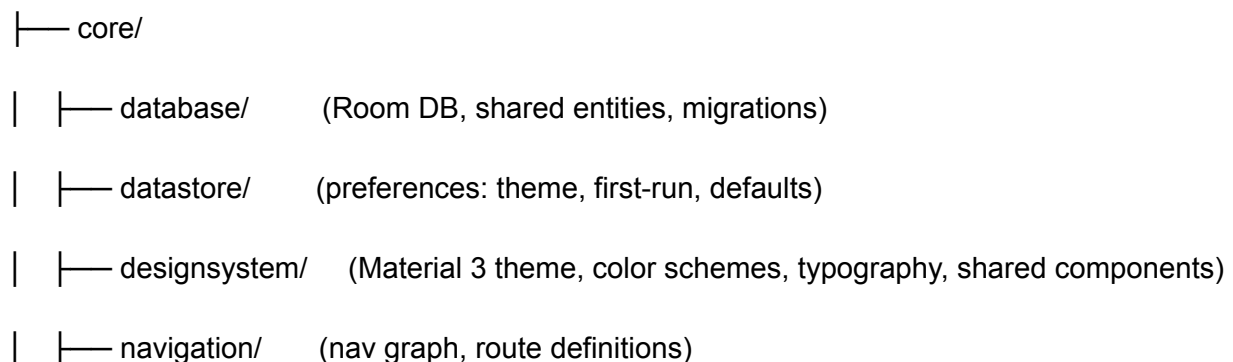




Why this matters for extensibility: New features (e.g., "Assignments") get their own Use Cases and Repository interface, without the Timetable feature's ViewModel or UI ever needing to know they exist. Nothing is hardwired to a single "God ViewModel" or a single giant database access class.

4.2 Package Structure

com.example.cadence



```

|   └─ util/          (date/time helpers, extension functions)
|
|   └─ feature/
|       └─ timetable/  (weekly grid, day view — CORE feature)
|           └─ data/
|               └─ domain/
|                   └─ presentation/
|                       └─ courses/      (course CRUD)
|                           └─ semesters/ (semester management)
|                               └─ settings/
|                                   └─ assignments/ (Phase 2 — exams/homework tracker)
|                                       └─ attendance/ (Phase 2)
|                                           └─ widgets/ (Glance widgets)
|                                               └─ backup/ (export/import, Phase 1 basic + Phase 2 cloud)
|
|   └─ CadenceApp.kt / MainActivity.kt

```

Each `feature/*` package is self-contained: its own data models, repository interface + impl, use cases, ViewModel(s), and Compose screens. This is the primary mechanism that lets you "improve or expand without much hassle" — adding a feature means adding a new package, not editing five existing files.

4.3 Data Flow Example (adding a class session)

User taps "Save" on Add Class screen

→ `ViewModel.onSaveClicked()`

- AddClassSessionUseCase(sessionData)
- ClassSessionRepository.insert(session)
- ClassSessionDao.insert(entity) [Room writes to disk]
- Flow<List<ClassSession>> automatically emits new data
- Timetable screen ViewModel collects Flow → UI recomposes

Because Room DAOs return **Flow**, the UI is always reactive to data changes — no manual refresh logic needed anywhere in the app.

5. Data Model

5.1 Entity-Relationship Overview

Semester (1) —< (many) Course

Course (1) —< (many) ClassSession

Course (1) —< (many) Assignment [Phase 2]

ClassSession (1) —< (many) AttendanceRecord [Phase 2]

5.2 Core Entities (Phase 1 — MVP)

Semester | Field | Type | Notes | |---|---|---| | id | Long (PK, autogen) | | | name | String | e.g. "Fall 2026" | | startDate | LocalDate | used for "week number" calculations | | endDate | LocalDate | | | isActive | Boolean | only one semester active at a time; drives what Home shows | | createdAt | Instant | |

Course | Field | Type | Notes | |---|---|---| | id | Long (PK) | | | semesterId | Long (FK → Semester) | | | name | String | e.g. "Intro to Algorithms" | | code | String? | e.g. "CS201" | | | teacherName | String? | | | teacherContact | String? | optional email/phone | | colorSeed | Int | used to derive a consistent Material color tag for this course across the UI | | credits | Float? | optional, useful for future GPA feature | | | notes | String? | free text |

ClassSession (a recurring weekly slot for a course) | Field | Type | Notes | |---|---|---| | id | Long (PK) | | | courseId | Long (FK → Course) | | | dayOfWeek | Int (1–7, ISO) | | | startTime |

LocalTime | | | endTime | LocalTime | | | room | String? | | | building | String? | separated from room to support future map linking | | sessionType | Enum: LECTURE, LAB, TUTORIAL, SEMINAR, OTHER | for color/icon differentiation | | recurrenceWeeks | Enum: ALL, ODD, EVEN | supports biweekly/alternating schedules | | notes | String? | |

ReminderSetting (per session, optional) | Field | Type | Notes | |---|---|---| | id | Long (PK) | | | classSessionId | Long (FK) | | | minutesBefore | Int | e.g. 15 | | enabled | Boolean | |

5.3 Phase 2 Entities (designed now, built later)

Assignment — id, courseId (FK), title, dueDate, dueTime, type (HOMEWORK/EXAM/PROJECT/QUIZ), completed (Boolean), notes.

AttendanceRecord — id, classSessionId (FK), date, status (PRESENT/ABSENT/EXCUSED/LATE).

Grade — id, courseId (FK), label (e.g., "Midterm"), weightPercent, scoreAchieved, scoreMax. *(Enables an optional GPA/grade calculator.)*

These are specified now purely so the Phase 1 schema doesn't need breaking changes later — foreign keys and ID strategy already accommodate them.

5.4 Schema Versioning & Migrations

- Room's `@Database(version = N)` is incremented on every schema change, with an explicit `Migration(oldVersion, newVersion)` object — never `fallbackToDestructiveMigration()` in production, since users' schedules are irreplaceable data.
- Each migration lives in `core/database/migrations/MigrationN_M.kt`, individually testable via Room's `MigrationTestHelper`.

5.5 Backup / Export Format

A JSON export (human-readable, versioned) allows manual backup/restore and future cloud sync without being tied to Room internals:

```
{  
  
  "exportVersion": 1,  
  
  "exportedAt": "2026-09-04T10:00:00Z",  
  
  "semesters": [ { "id": 1, "name": "Fall 2026", "courses": [ ... ] } ]  
}
```


}

This also becomes the transport format for a future "share your timetable" or "duplicate to next semester" feature.

6. Feature Specification

6.1 Phase 1 — Core Features (MVP)

F1. Semester Management

- Create a semester (name, start date, end date).
- Mark one semester "active" — this drives what the Home/Week views display.
- Archive semesters (soft-delete: hidden from active list, data retained, viewable in "Past Semesters").
- Duplicate a semester's course structure into a new semester (huge time-saver for recurring schedules) — clones courses/sessions with new IDs, blank of any Phase 2 attendance/grade data.

F2. Course Management (CRUD)

- Add/edit/delete a course: name, code, teacher, color, credits, notes.
- Color is either auto-assigned (rotating Material palette) or manually picked from a curated Material You–derived swatch set.

F3. Class Session Management (CRUD)

- Add/edit/delete a recurring weekly session tied to a course: day, start/end time, room, building, session type, recurrence (all/odd/even weeks).
- Conflict detection: warns (non-blocking) if a new session overlaps an existing one in time.

F4. Timetable Views

- **Week Grid View:** classic timetable grid — days as columns, time as vertical axis, each class rendered as a colored block sized to duration.
- **Day Agenda View:** vertical list of today's (or selected day's) classes in chronological order — simpler, more readable on narrow screens, good default for phones.
- Toggle between the two; user preference remembered.

- **Today / Home View:** glanceable "what's now, what's next" card at the top, followed by the rest of today's agenda.

F5. Class Detail

- Tapping a class opens a detail sheet: course name, teacher, room/building, time, session type, notes, quick-edit and delete actions.

F6. Search

- Simple search/filter across courses and sessions (by course name, code, teacher, or room).

F7. Notifications & Reminders

- Optional per-session reminder (e.g., 15 min before class) via WorkManager + system notification.
- Uses a dedicated notification channel ("Class Reminders") so the user can control it in system settings.

F8. Theming

- Full Material You dynamic color (derives from wallpaper on Android 12+).
- Manual override: Light / Dark / System / "Static color" fallback (for users who don't want wallpaper-based theming, or on devices where dynamic color is unavailable).

F9. Backup & Restore

- Manual export to a JSON file (via Storage Access Framework — user picks location, no broad storage permission needed).
- Manual import/restore from a previously exported file.

F10. Onboarding

- First-run flow: create your first semester → add your first course → (optionally) add your first class session, or skip and explore an empty state with clear "Add" prompts.

6.2 Phase 2 — Expanded / Modern Features

These are scoped and designed for, but not required for MVP launch. Each is additive and isolated to its own feature package (Section 4.2).

F11. Home Screen Widgets (Glance API)

- "Next Class" widget — small widget showing the very next upcoming class, countdown, and room.
- "Today's Schedule" widget — medium/large widget listing the day's remaining classes.
- Widgets use the same Room data via a shared repository — no duplicated logic.

F12. App Shortcuts

- Long-press app icon → dynamic shortcuts: "Today's Schedule," "Add Class," "Next Class."
- Implemented via `ShortcutManagerCompat` — static shortcuts for fixed actions, dynamic for "jump to today."

F13. Quick Settings Tile

- A QS tile ("Next Class") the user can add to their notification shade for a one-glance check without opening the app.

F14. Assignment & Exam Tracker

- Tied to `Assignment` entity (Section 5.3). Due-date list, sortable, with reminders (reusing the same `WorkManager`/notification infrastructure as class reminders).
- Optional linking to a course for color-coding and cross-display in the course detail screen.

F15. Attendance Tracker

- Mark present/absent/excused per class occurrence.
- Simple attendance percentage displayed per course (useful for courses with mandatory-attendance policies).

F16. Grade / GPA Calculator (optional module)

- Weighted grade entries per course; computed running grade and optional overall GPA if the user enters credit hours and a grading scale.
- Entirely optional — a course with no grade entries simply doesn't show this section.

F17. Calendar Export / Sync

- One-way export of the semester's recurring sessions to the device's Calendar Provider (`CalendarContract`) or as a standard `.ics` file — lets users see classes in Google Calendar without making Cadence dependent on any calendar account.

F18. Room/Building Location Assist

- Optional field linking a building name to a map query (opens Google Maps / Geo intent) — no in-app maps SDK needed, keeps the app lightweight and offline-capable.

F19. Share / Export Timetable as Image or PDF

- Renders the current week grid to a shareable PNG or PDF (useful for sharing with classmates or printing).

F20. Adaptive Layouts (Tablet / Foldable)

- Two-pane layout on wide screens: course list + detail side-by-side; full week grid with more visible columns.
- Built using `WindowSizeClass` from the start in Compose layout logic so this is a styling change, not a rewrite.

F21. Optional Cloud Backup (stretch goal)

- Explicitly opt-in Google Drive backup of the export JSON (Section 5.5) via the Drive REST API or `SAF` + user's own cloud provider app — never a requirement, never automatic.

F22. Accessibility & Localization

- Full TalkBack labeling, minimum 48dp touch targets, dynamic font scaling support, and all UI strings externalized to `strings.xml` for future translation.

7. Screen-by-Screen UX Specification

7.1 Onboarding

- **Screen 1 — Welcome:** brief value prop, "Get Started" CTA.
- **Screen 2 — Create Semester:** name + date range fields, sensible defaults (e.g., name pre-filled as "Fall 2026" based on current date).
- **Screen 3 — Add First Course (optional/skippable):** minimal form (name + color), "Add class times now" or "I'll do this later."
- Ends on the **Home** screen, empty state if skipped, with a prominent FAB prompting "Add your first class."

7.2 Home / Today Screen

- Top: dynamic greeting + current date + active semester name.
- "Happening now" card (if a class is currently in session) — large, prominent, shows countdown to end.
- "Up next" card — the following class with countdown to start.
- Below: scrollable agenda of the rest of today's classes.
- Empty state (no classes today): friendly illustration/message, e.g. "No classes today — enjoy the break."
- FAB: quick-add class session.

7.3 Week View

- Segmented control or tab: **Grid / Agenda**.
- **Grid mode**: horizontally scrollable if needed on small screens; days as column headers, time gridlines, color-coded class blocks sized by duration; tapping a block opens Class Detail bottom sheet.
- **Agenda mode**: day-by-day vertical sections (Mon, Tue, ...), each listing that day's classes as Material **Cards**.
- Week navigation: previous/next week arrows (relevant for odd/even recurrence display) + "Jump to this week."

7.4 Course List / Management

- List of all courses in the active semester as cards (color swatch, name, code, teacher, session count).
- Tap → Course Detail (shows all sessions for that course, plus Phase 2 tabs for Assignments/Attendance/Grades if enabled).
- FAB → Add Course.
- Swipe-to-archive or delete with confirmation + undo snackbar.

7.5 Add/Edit Course

- Form fields: Name*, Code, Teacher name, Teacher contact (optional), Color picker (Material palette swatches), Credits (optional), Notes.
- Validation: Name required; inline error state, not blocking dialogs.

7.6 Add/Edit Class Session

- Course picker (dropdown/search, or launched from within Course Detail with course pre-filled).
- Day picker (single-select chip row: Mon–Sun).
- Start/End time pickers (Material 3 time picker).
- Room + Building fields.

- Session type chips (Lecture/Lab/Tutorial/Seminar/Other).
- Recurrence selector (All weeks / Odd weeks / Even weeks).
- Reminder toggle + minutes-before selector.
- Conflict warning banner if overlapping an existing session (non-blocking, dismissible).

7.7 Class Detail (Bottom Sheet)

- Course name + color tag, teacher, time range, room/building (tappable → map intent if F18 enabled), session type badge, notes.
- Actions: Edit, Delete (confirm), and (Phase 2) Mark Attendance.

7.8 Semester Management

- List of semesters (Active first, then chronological past semesters).
- Actions per semester: Set Active, Duplicate, Archive/Unarchive, Delete (confirm, irreversible).
- "New Semester" CTA at top.

7.9 Settings

- Theme: Light / Dark / System / Dynamic Color toggle.
- Notification defaults (default reminder lead time).
- Backup & Restore (export/import buttons, last backup timestamp shown).
- Data management (clear archived semesters, storage info).
- About (version, licenses).

7.10 Search

- Accessible from top app bar icon; live-filters courses/sessions as the user types; tapping a result jumps to Class Detail or Course Detail.

7.11 Widget Configuration (Phase 2)

- Standard Android widget picker → optional configuration Activity to choose which semester/course subset to display (defaults to active semester, all courses).

8. Navigation Structure

- **Bottom Navigation Bar** (Material 3 [NavigationBar](#)), 4 primary destinations:
 1. **Today** (home icon)
 2. **Week** (calendar/grid icon)
 3. **Courses** (book icon)
 4. **Settings** (gear icon)

- Semester management is accessed from Settings (or a top-bar dropdown showing the active semester name, tappable to switch/manage — recommended, since semester context matters across all screens).
- Search is a top-app-bar action available from Today/Week/Courses.
- Add flows (Add Course, Add Class Session) are modal full-screen or bottom-sheet destinations reached via FAB, not part of the bottom nav.
- On tablets/foldables (F20): bottom nav becomes a `NavigationRail` on the side; list+detail screens use two-pane `ListDetailPaneScaffold` (Material 3 adaptive library).

Implemented via a single `NavHost` with sealed-class `Route` definitions in `core/navigation`, so adding a new top-level destination (e.g., "Assignments" tab in Phase 2) is a one-file change plus one new bottom-nav item.

9. Design System — Material You Application

9.1 Color

- `dynamicLightColorScheme(context)` / `dynamicDarkColorScheme(context)` used when available (Android 12+, which is our whole target range, so this is always-on by default) with a manual static Material 3 baseline palette as fallback/opt-out for users who disable dynamic color in Settings.
- Course color tags are derived by mapping each course's `colorSeed` to a curated set of `tertiary/secondary` container colors from the current scheme — so course colors always stay harmonious with the user's system theme rather than being arbitrary hardcoded hues.

9.2 Typography

- Material 3 type scale (`displayLarge` → `labelSmall`) applied via a single `Typography` object in `core/designsystem`. Headlines for screen titles, `titleMedium` for card headers, `bodyMedium` for details, `labelSmall` for metadata (room, time).

9.3 Shape

- Material 3 shape scale: small radius for chips/badges, medium for cards (class blocks, course cards), large for bottom sheets and dialogs.

9.4 Components

- `Card` (elevated/filled variants) for class blocks and course items.

- `FilterChip` for session type and day selection.
- `NavigationBar` / `NavigationRail` for primary nav.
- `ModalBottomSheet` for Class Detail and quick-add flows.
- `FloatingActionButton` (standard + extended variants) for primary add actions.
- `SegmentedButton` for Grid/Agenda toggle.
- `TimePicker` / `DatePicker` (Material 3 dialogs) for all time/date input.

9.5 Motion

- Shared-element-style transitions between Week grid blocks and Class Detail sheet (Compose `SharedTransitionLayout` where feasible).
- Standard Material motion easing/duration tokens for screen transitions (no custom animation curves needed).
- **Predictive Back** gesture support (Android 13+ API, gracefully degrades to standard back on Android 12) for all modal/detail screens.

9.6 Iconography

- Material Symbols (outlined style default, filled for selected/active nav states).

10. Android Platform Integration Details

Feature	API / Mechanism	Notes
Dynamic color	<code>dynamicLightColorScheme</code> / <code>dynamicDarkColorScheme</code> (Android 12+)	Always available on our min SDK — a core differentiator, not a bolt-on.
Reminders	<code>WorkManager</code> (<code>OneTimeWorkRequest</code> scheduled per session per week) + <code>NotificationCompat</code>	Avoids <code>AlarmManager</code> exact-alarm complexity/permissions where inexact timing (a few minutes) is acceptable; escalate to exact alarms only if user feedback demands precision.
Notification permission	<code>POST_NOTIFICATIONS</code> runtime permission (required)	Requested contextually — e.g., when the user first

Feature	API / Mechanism	Notes
	Android 13+, harmless request on 12)	enables a reminder, not on app launch.
Widgets	Glance API (<code>GlanceAppWidget</code>)	Compose-first widget authoring, shares ViewModels/repositories with the main app.
App Shortcuts	<code>ShortcutManagerCompat</code>	Static shortcuts declared in XML for fixed actions; dynamic shortcut updated to reflect "today" context.
Quick Settings Tile	<code>TileService</code>	Optional Phase 2 feature.
File export/import	Storage Access Framework (<code>ACTION_CREATE_DOCUMENT</code> / <code>ACTION_OPEN_DOCUMENT</code>)	No <code>WRITE_EXTERNAL_STORAGE</code> permission needed at all.
Calendar export	<code>CalendarContract</code> provider or <code>.ics</code> file generation	User consents per-export; no persistent calendar permission held.
Predictive back	<code>onBackPressedCallback</code> / <code>Compose PredictiveBackHandler</code>	Android 13+ enhancement, degrades gracefully on 12.
Adaptive layouts	<code>WindowSizeClass</code> (Material3 adaptive/ <code>androidx.window</code>)	Drives phone vs. tablet/foldable layout branching.
Auto Backup for Apps	Standard Android manifest flag, with explicit <code>backup_rules.xml</code>	Room DB included by default so users regain data on device restore even without manual export — complements, not replaces, F9.

11. Non-Functional Requirements

11.1 Performance

- Cold start under ~1.5s on a mid-range Android 12 device.
- Timetable grid renders smoothly (60fps target) even with 15+ courses/sessions — achieved via `LazyLayout`-based grid rendering, not a fully custom heavyweight `Canvas` unless profiling proves it necessary.

11.2 Offline Guarantee

- Zero network calls required for any Phase 1 feature. Phase 2's calendar export and optional cloud backup are the only features that ever touch the network, and both require explicit user action.

11.3 Accessibility

- All interactive elements meet 48dp minimum touch target.
- Full TalkBack content descriptions on icons, color-only information (course color tags) always paired with text.
- Respects system font-scale and "reduce motion" accessibility settings.

11.4 Privacy & Security

- No analytics/telemetry by default. If added later, must be explicitly opt-in and disclosed.
- All data stored locally in the app's private Room database (sandboxed by Android per-app storage).
- Exported backup files are plain JSON the user controls — Cadence does not transmit them anywhere itself.

11.5 Testing Strategy

- **Unit tests:** domain use cases (pure Kotlin, fast), ViewModels (with fake repositories).
 - **Database tests:** Room DAOs tested with in-memory database; migrations tested with `MigrationTestHelper`.
 - **UI tests:** Compose UI testing for critical flows (add course → add session → appears in Week view; create semester → switch active semester).
 - **Manual QA checklist:** dynamic color across multiple wallpapers, dark/light toggle, rotation/foldable states, TalkBack pass.
-

12. Extensibility & Roadmap

12.1 How This Spec Supports Painless Growth

1. **Feature-package isolation** (Section 4.2) — new features are new packages, not edits scattered across existing ones.
2. **Repository-interface pattern** — UI/domain layers depend on interfaces, not concrete Room implementations, so a future data source (e.g., cloud sync) can be swapped in behind the same contract.
3. **Schema designed ahead** (Section 5.3) — Phase 2 entities' foreign keys are already compatible with Phase 1 tables, avoiding painful migrations later.
4. **Adaptive-from-day-one layout logic** (`WindowSizeClass`) — tablet/foldable support is a layout branch, not a retrofit.
5. **Versioned export format** — a stable contract for backup/restore, sharing, and any future sync feature to build on.

12.2 Suggested Build Phases

Phase	Scope
Phase 1 (MVP)	F1–F10: full offline timetable app — semesters, courses, sessions, week/day views, reminders, theming, backup/restore.
Phase 2a	F11–F13: widgets, shortcuts, QS tile — highest "delight per effort" additions using existing data only.
Phase 2b	F14–F16: assignments, attendance, grades — new entities, new feature packages.
Phase 2c	F17–F19: calendar export, location assist, share/export image.
Phase 3	F20–F22: adaptive tablet/foldable layouts, optional cloud backup, full accessibility/localization polish.

12.3 Open Questions to Resolve Before Build

- Exact reminder mechanism precision needed (`WorkManager`'s inexact timing vs. exact alarms) — depends on how critical "exactly 15 minutes before" is to you.

- Whether course color should be user-pickable from day one or auto-assigned only in Phase 1 (recommend: auto-assign in Phase 1, add manual override in Phase 2 to keep MVP scope tight).
 - Whether biweekly (odd/even week) recurrence is common enough at your institution to justify in Phase 1, or can be deferred.
-

13. Appendix

13.1 Glossary

- **Session:** a single recurring weekly time slot for a course (e.g., "CS201 Lecture, Mon 10–11am").
- **Active semester:** the one semester currently driving Home/Week views.
- **Dynamic color:** Android 12+ system feature deriving an app's color scheme from the user's wallpaper.

13.2 Full Tech Stack Summary

Kotlin · Jetpack Compose · Material 3 · Room · Hilt · Coroutines/Flow · Navigation Compose · DataStore · WorkManager · Glance · kotlinx-datetime.

End of specification.